

lab10

May 1, 2026

By: Rafay Siddiqui
Student No: 24K-0009
Section: BAI-4A

1 Q1

Imagine you're a data scientist working for a real estate company. Your task is to build a model to predict house prices based on various features such as the number of bedrooms, square footage, and the location of the house. You have access to a dataset with information about houses, including their square footage, number of bedrooms, number of bathrooms, age of the house, and location (neighborhood). The goal is to predict the price of the house, which is a continuous variable. Perform the following

task: - Clean the dataset and handle any missing values. Encode categorical variables (e.g., neighborhood) as numeric values. - Identify the relevant features that most likely impact the price. - Evaluate the performance of the model using any metrics. - Predict the price of a house given a new set of features.

```
[ ]: import pandas as pd
import numpy as np

houses_data = {
    'sq_ft': [1200, 2500, 1800, 4500, 1100, 3200, 1500, 2200, 5000, 2800],
    'bedrooms': [2, 4, 3, 5, 2, 4, 3, 3, 6, 4],
    'bathrooms': [2, 3, 2, 4, 1, 3, 2, 2, 5, 3],
    'age': [15, 5, 10, 2, 20, 8, 12, 6, 1, 7],
    'neighborhood': ['Gulshan', 'DHA', 'Nazimabad', 'Clifton', 'Gulshan',
                     'DHA', 'Nazimabad', 'Clifton', 'DHA', 'Gulshan'],
    'price': [12000000, 35000000, 18000000, 85000000, 11000000,
              45000000, 16000000, 38000000, 11000000, 30000000]
}

pd.DataFrame(houses_data).to_csv('houses.csv')
print("houses.csv generated.")
```

```
[ ]: import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
```

```

from sklearn.metrics import mean_squared_error

df = pd.read_csv('houses.csv')
df.fillna(df.median(numeric_only=True), inplace=True)
df = pd.get_dummies(df, drop_first=True)

X = df.drop('price', axis=1)
y = df['price']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
model = LinearRegression().fit(X_train, y_train)

print("MSE:", mean_squared_error(y_test, model.predict(X_test)))
print("Prediction:", model.predict(X_test.iloc[[0]]))

```

2 Q2

You work as a data scientist for an email service provider. Your task is to develop a model that can classify emails as spam or not spam based on their content. You have a labeled dataset with thousands of emails. Each email has features such as frequency of specific words, length of the email, presence of hyperlinks, and sender's address. The task is to classify an email as spam (1) or not spam (0). - Preprocess the dataset by converting text features to numerical features. - Train a model to classify the emails based on their features. - Evaluate the model's performance. - Deploy the model to classify new incoming emails.

```

[ ]: emails_data = {
    'content': [
        "Hey, are we still meeting for biryani today?",
        "URGENT: Your bank account has been locked. Click here to verify.",
        "Congratulations! You've won a luxury trip to Dubai. Call now!",
        "Please find the attached assignment for the Data Science lab.",
        "Get rich quick! Work from home and earn 5000 USD a day.",
        "Reminder: FAST NUCES fee deadline is tomorrow.",
        "Free entry into our $1000 prize draw. Text WIN to 8000.",
        "Can you send me the notes for the linear algebra quiz?",
        "Your package is out for delivery. Track it on our website.",
        "Double your crypto earnings in just two hours! Guaranteed."
    ],
    'is_spam': [0, 1, 1, 0, 1, 0, 1, 0, 0, 1]
}

pd.DataFrame(emails_data).to_csv('emails.csv')
print("emails.csv generated.")

```

```

[ ]: import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split

```

```

from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score

df = pd.read_csv('emails.csv')

vec = TfidfVectorizer()
X = vec.fit_transform(df['content'])
y = df['is_spam']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
model = MultinomialNB().fit(X_train, y_train)

print("Accuracy:", accuracy_score(y_test, model.predict(X_test)))
print("Prediction:", model.predict(vec.transform(["Congratulations! You won a_
↳free gift card. Click here."])))

```

3 Q3

You work for a retail business and your task is to classify customers into two categories: high-value and low-value customers. The classification is based on customer features such as total spending in the last 6 months, age, number of visits, and purchase frequency. You have a dataset with customer information, including spending habits, frequency of visits, and demographics. The goal is to classify customers into high-value (1) and low-value (0) categories. - Clean the dataset and handle any missing or outlier values. Perform feature scaling if necessary. - Divide the dataset into a training set and a testing set. - Find a separating hyperplane for classification. - Find rules that classify customers based on their features. - Evaluate the performance of the model.

```

[ ]: customers_data = {
    'spending_6m': [5000, 450, 8000, 200, 12000, 300, 7500, 600, 15000, 100,
↳9999999], # Last one is an outlier
    'age': [35, 19, 42, 21, 38, 20, 45, 23, 50, 18, 25],
    'visits': [15, 2, 20, 1, 25, 3, 18, 4, 30, 1, 100],
    'frequency': [5, 1, 8, 1, 10, 1, 7, 2, 12, 1, 50],
    'high_value': [1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1]
}

pd.DataFrame(customers_data).to_csv('customers.csv')
print("customers.csv generated.")

```

```

[ ]: import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.metrics import accuracy_score
from scipy.stats import zscore

```

```

df = pd.read_csv('customers.csv')
df.fillna(df.median(), inplace=True)
df = df[(np.abs(zscore(df.select_dtypes(include=np.number))) < 3).all(axis=1)]

X = df.drop('high_value', axis=1)
y = df['high_value']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

svm = SVC(kernel='linear').fit(X_train_scaled, y_train)
dt = DecisionTreeClassifier(max_depth=3).fit(X_train, y_train)

print("SVM Accuracy:", accuracy_score(y_test, svm.predict(X_test_scaled)))
print("Tree Accuracy:", accuracy_score(y_test, dt.predict(X_test)))
print("Classification Rules:\n", export_text(dt, feature_names=list(X.columns)))

```